Roteiro — Aula de Backend (3h30)

Projeto: ENCOMP Agenda · API REST de eventos
Stack: Node.js · Express · SQLite
Arquitetura: `index.js` + `db.js` + `routes/eventos.js`

Objetivo

Construir juntos uma API CRUD, estudando **separação de responsabilidades**, e publicar no Render (instrutor).

Pré-requisitos

Alunos

- [ ] Node.js LTS — `node -v` / `npm -v`
- [ ] VS Code + Thunder Client
- [ ] Git e navegador

Instrutor

- [ ] Conta Render + slides
- [ ] Ao final: URL no quadro

Alunos **não** precisam de Render.

Bloco 1 — Teoria (0:00–0:15)

Slides `slides-backend.pdf`.

- API, HTTP, JSON, REST, modelo `evento`
- Arquitetura em 3 arquivos (entrada / banco / rotas)
- Demo no Thunder Client

Depois: prática.

Bloco 2 — Express + health (0:15–0:40)

- `npm init -y` · `npm i express cors better-sqlite3`
- `index.js`: app, `cors`, `json`, `GET /health`, `listen`

Saída: servidor local ok.

Bloco 3 — Banco (`db.js`) (0:40–1:05)

- criar `db.js`: pasta `data`, `agenda.db`, `CREATE TABLE`, seed
- `require("./db")` no `index.js` (ou só via rotas)
- mostrar o arquivo `.db` gerado

Saída: banco criado.

Bloco 4 — Rotas: listar e criar (1:05–1:50)

- criar `routes/eventos.js` com `Router`
- `GET /` e `POST /`
- em `index.js`: `app.use("/eventos", eventosRouter)`

- validação simples no POST

Saída: listar e criar no Thunder Client.

Bloco 5 — Atualizar e excluir (1:50–2:25)

- `PUT /:id` e `DELETE /:id` em `routes/eventos.js`
- 404 se não achar

Saída: CRUD completo.

Bloco 6 — Revisar arquitetura (2:25–2:40)

- quem faz o quê: `index` / `db` / `routes`
- `PORT` e `.gitignore`

Bloco 7 — Deploy instrutor (2:40–3:30)

- Render · root `backend` · `npm start`
- URL no quadro

Entrega: API no ar.

Checklist

- [ ] `/health` público
- [ ] CRUD `/eventos` na URL do Render
- [ ] URL no quadro

Cola

Método	Caminho
GET	`/health`
GET	`/eventos`
POST	`/eventos`
PUT	`/eventos/:id`
DELETE	`/eventos/:id`